Fatec Itapetininga - Virtual

ARQUITETURA CLOUD-NATIVE PARA SISTEMAS ESCALÁVEIS: INTEGRAÇÃO DE MICROSSERVIÇOS, SERVERLESS E OBSERVABILIDADE
CLOUD-NATIVE ARCHITECTURE FOR SCALABLE SYSTEMS: INTEGRATING MICROSERVICES, SERVERLESS COMPUTING, AND OBSERVABILITY

RESUMO: Propõe-se uma arquitetura cloud-native para sistemas escaláveis, combinando microsserviços, serverless, contêineres, persistência poliglota e observabilidade. O estudo é exploratório e apresenta decisões arquiteturais e um plano de validação, sem resultados experimentais.

Palavras-chave: Elasticidade; Gestão de custos; Persistência poliglota; Sistemas distribuídos; Tolerância a falhas.

ABSTRACT: This paper proposes a cloud-native architecture for scalable systems combining microservices, serverless, containers, polyglot persistence, and observability. The exploratory study presents architectural decisions and a validation plan, without experimental results.

Keywords: Cost management; Distributed systems; Elasticity; Fault tolerance; Polyglot persistence.

1 INTRODUÇÃO

A expansão de serviços digitais exige sistemas que suportem variações de carga, integração entre componentes e evolução frequente. A adoção de nuvem não garante essas propriedades: elas dependem de decisões sobre fronteiras dos serviços, comunicação, dados e operação. A literatura associa microsserviços à evolução independente, mas também registra custos de integração, testes e observabilidade distribuída (Newman, 2021; Di Francesco et al., 2019).

O problema investigado é como estruturar uma arquitetura em nuvem que favoreça evolução e escalabilidade sem transferir complexidade excessiva para operação, segurança e monitoramento. O objetivo é propor uma solução cloud-native que relacione requisitos não funcionais a componentes, responsabilidades e tecnologias, combinando execução serverless e em contêineres, comunicação assíncrona, persistência, segurança e observabilidade.

A questão de pesquisa é: como combinar serviços gerenciados, execução serverless e microsserviços em contêineres preservando evolução, segurança, observabilidade e controle de custos? A pesquisa é aplicada, exploratória e ainda está em fase de modelagem.

Assim, o manuscrito apresenta decisões e plano de validação, sem afirmar resultados de desempenho, disponibilidade ou economia que não foram medidos.

2 METODOLOGIA

O estudo combina levantamento bibliográfico exploratório, análise documental dos artefatos do projeto e modelagem arquitetural. Foram priorizadas fontes sobre microsserviços, serverless, contêineres, segurança e arquitetura em nuvem, incluindo trabalhos científicos, livros técnicos e documentação oficial. As fontes foram utilizadas para identificar benefícios, limitações e condições de uso, e não apenas para reunir definições.

Os objetivos foram relacionados aos requisitos de escalabilidade, disponibilidade, manutenibilidade, segurança, observabilidade e custo. Em seguida, esses requisitos orientaram a definição de componentes, responsabilidades, fluxos síncronos e assíncronos, limites de confiança e alternativas tecnológicas. O resultado foi registrado em diagramas Mermaid, descrição textual e matriz de decisões no repositório.

A verificação desta etapa é documental. Não foram implementados protótipo, testes de carga ou experimentos com usuários; portanto, a ausência de dados impede conclusões empíricas. A validação futura deverá medir latência, throughput, taxa de erro, comportamento sob falhas, tempo de fila e custo em cenários controlados. A revisão não sistemática e a escolha inicial de serviços AWS também limitam a generalização da proposta.

3 REFERENCIAL TEÓRICO

3.1 Microsserviços e fronteiras de responsabilidade

Microsserviços organizam uma aplicação em serviços com responsabilidades delimitadas e possibilidade de implantação independente. As fronteiras devem refletir capacidades do domínio, evitando distribuir um monólito sem reduzir o acoplamento (Newman, 2021). Um mapeamento sistemático aponta benefícios de evolução e escalabilidade seletivas, acompanhados de desafios de integração, testes e observabilidade (Di Francesco et al., 2019). A proposta usa responsabilidade de negócio e estabilidade de interfaces para decidir a divisão dos componentes.

3.2 Computação serverless e contêineres

Serverless delega ao provedor parte da infraestrutura e executa funções sob demanda. Revisões descrevem adequação a eventos e cargas variáveis, mas também limitações de inicialização, duração, depuração, portabilidade e dependência do provedor (Shafiei et al., 2019; John & Gupta, 2021; Li et al., 2021; Wen et al., 2022). Uma análise de cargas reais de FaaS mostra que frequência de invocação e cold starts afetam diretamente o gerenciamento de recursos (Shahrad et al., 2020). Estudos sobre impacto econômico e carregamento de contêineres indicam que o modelo deve ser escolhido conforme o perfil de cada componente (Adzic & Chatley, 2017; Brooker et al., 2023). Por isso, a arquitetura adota um modelo híbrido, com Lambda para tarefas curtas e contêineres para serviços que exigem maior controle do runtime.

3.3 Segurança, observabilidade e operação

Filas e tópicos podem reduzir dependências temporais e absorver variações de demanda, desde que sejam tratados duplicidade, idempotência, retentativas e mensagens não processáveis. Em sistemas distribuídos, autenticação, autorização, criptografia, isolamento, logs, métricas e rastreamento devem acompanhar os fluxos desde o desenho (Indrasiri & Siriwardena, 2020; AWS, n.d.-e). Logs estruturados permitem reconstruir execuções, métricas mostram tendências e rastreamento relaciona uma requisição aos componentes percorridos.

3.4 Trabalhos relacionados e critérios de decisão

Os estudos revisados convergem que modularidade e elasticidade podem favorecer a evolução, mas introduzem custos de integração, monitoramento, portabilidade e governança. Adzic e Chatley (2017) discutem o impacto econômico do serverless, enquanto Brooker et al. (2023) analisam carregamento sob demanda de contêineres. A proposta sintetiza cinco critérios para selecionar o modelo de execução: responsabilidade de negócio, perfil de carga, necessidade de estado, criticidade do dado e exigência de diagnóstico.

4 RESULTADOS E DISCUSSÃO

4.1 Visão geral da solução

Os artefatos do projeto descrevem componentes, fluxos e decisões. A arquitetura inclui aplicação web em S3 e CloudFront, API Gateway e Cognito, processamento em Lambda e ECS/Fargate, PostgreSQL e DynamoDB, mensageria SQS/SNS e telemetria via CloudWatch e rastreamento distribuído (AWS, n.d.-a, n.d.-b, n.d.-c, n.d.-d).

O fluxo começa no navegador. O conteúdo estático é distribuído pelo CloudFront a partir do S3, enquanto chamadas de negócio seguem para o API Gateway. O gateway funciona como fronteira de entrada, aplica limites e encaminha cada operação para a função ou serviço correspondente. O Cognito concentra a identidade e fornece o contexto de autorização aos componentes internos.

4.2 Componentes e responsabilidades

Lambda atende eventos, integrações e operações curtas. ECS/Fargate atende serviços de domínio com execução mais longa, dependências específicas ou necessidade de controle do processo. O RDS PostgreSQL é indicado para entidades relacionais e transacionais; o DynamoDB, para dados com chaves de acesso conhecidas e alta elasticidade. A persistência poliglota exige sistema de referência, regras de sincronização e política de recuperação para não deslocar a complexidade para a aplicação.

4.3 Comunicação e comportamento sob falhas

Operações imediatas podem usar o caminho síncrono entre gateway e serviço. Processamentos demorados, notificações e integrações devem ser publicados em tópicos ou filas. Produtores devem registrar identificadores idempotentes; consumidores devem tolerar duplicidade, aplicar retentativas controladas e encaminhar mensagens não processáveis para uma fila de análise. A consistência eventual deve ser explicitada ao usuário quando o processamento ainda estiver pendente.

4.4 Segurança, escalabilidade e observabilidade

O acesso público deve usar HTTPS, gateway e políticas de autenticação. Recursos de persistência não devem ser expostos diretamente; funções e contêineres devem utilizar menor privilégio, criptografia e, quando necessário, sub-redes privadas. Registros devem

remover segredos e dados pessoais desnecessários. Lambda, DynamoDB e ECS/Fargate podem acompanhar variações de demanda, enquanto RDS exige planejamento de capacidade, réplicas e limites de conexão. CloudWatch e X-Ray apoiam o diagnóstico, mas não substituem revisão de configuração e testes de ameaça.

4.5 Matriz de decisões

O Quadro 1 relaciona requisitos, decisões e verificações futuras. A avaliação deverá considerar segurança e custo além de escalabilidade (Di Francesco et al., 2019; AWS, n.d.-e).

Quadro 1 – Requisitos, decisões arquiteturais e validação prevista

Requisito	**Decisão proposta**	**Verificação futura**
Escalabilidade	Lambda e ECS/Fargate conforme a carga	Medir latência, throughput, erros e recursos.
Segurança	Cognito, tokens, TLS, IAM e isolamento	Revisar configuração, permissões e ameaças.
Persistência	RDS PostgreSQL e DynamoDB conforme os acessos	Avaliar consistência, concorrência e integridade.
Observabilidade	Logs, métricas, alertas e rastreamento	Injetar falhas e verificar rastreabilidade.
Resiliência	SQS/SNS, retentativas, backoff e idempotência	Exercitar duplicidade e reprocessamento.

Fonte: Elaborado pelos autores com base nos artefatos de arquitetura do projeto.

4.6 Plano de validação e limitações

A próxima etapa deve implementar um protótipo com um fluxo síncrono, um assíncrono, uma função Lambda, um serviço em contêiner e os dois padrões de persistência. A avaliação deverá comparar carga nominal e pico, registrar P50, P95 e P99, erros, throughput, tempo de fila, recursos e custos, além de provocar falhas controladas. Uma melhoria em cenário sintético não prova superioridade geral; configuração, volume, duração e limitações devem ser registrados para permitir reprodução.

5 CONSIDERAÇÕES FINAIS

A pesquisa organizou uma proposta cloud-native e explicitou a relação entre problema, requisitos, componentes, riscos e verificações futuras. Microsserviços favorecem limites de evolução, mas ampliam interações; serverless reduz a administração de servidores, mas pode aumentar dependência do provedor; persistência híbrida atende perfis distintos, mas requer governança.

Como não há protótipo nem validação empírica, escalabilidade, resiliência e custo permanecem hipóteses. Os próximos passos são revisar a arquitetura com os orientadores, implementar a prova de conceito, executar o protocolo de avaliação e atualizar o manuscrito com resultados auditáveis.

DECLARAÇÃO DE USO DE INTELIGÊNCIA ARTIFICIAL

“Ferramentas de inteligência artificial generativa, incluindo o ChatGPT (OpenAI), foram utilizadas como apoio à organização e à revisão linguística do manuscrito. Os autores são responsáveis pela seleção das fontes, conferência das referências, revisão do conteúdo e validação das afirmações.”

REFERÊNCIAS BIBLIOGRÁFICAS

Adzic, G., & Chatley, R. (2017). Serverless computing: Economic and architectural impact. In Proceedings of the 2017 11th Joint Meeting on Foundations of Software Engineering (pp. 884–889). ACM. https://doi.org/10.1145/3106237.3117767

AWS. (n.d.-a). Amazon API Gateway developer guide. Retrieved September 17, 2026, from https://docs.aws.amazon.com/apigateway/

AWS. (n.d.-b). Amazon CloudFront developer guide. Retrieved September 17, 2026, from https://docs.aws.amazon.com/cloudfront/

AWS. (n.d.-c). Amazon Cognito developer guide. Retrieved September 17, 2026, from https://docs.aws.amazon.com/cognito/

AWS. (n.d.-d). AWS Lambda developer guide. Retrieved September 17, 2026, from https://docs.aws.amazon.com/lambda/

AWS. (n.d.-e). AWS Well-Architected Framework. Retrieved September 17, 2026, from https://docs.aws.amazon.com/wellarchitected/

Brooker, M., Danilov, M., Greenwood, C., & Piwonka, P. (2023). On-demand container loading in AWS Lambda. arXiv. https://arxiv.org/abs/2305.13162

Di Francesco, P., Lago, P., & Malavolta, I. (2019). Architecting with microservices: A systematic mapping study. The Journal of Systems and Software, 150, 77–97. https://doi.org/10.1016/j.jss.2019.01.001

Indrasiri, K., & Siriwardena, P. (2020). Microservices security in action. Manning Publications.

John, J., & Gupta, S. (2021). A survey on serverless computing. arXiv. https://arxiv.org/abs/2106.11773

Li, Z., Guo, L., Cheng, J., Chen, Q., He, B., & Guo, M. (2021). The serverless computing survey: A technical primer for design architecture. arXiv. https://arxiv.org/abs/2112.12921

Newman, S. (2021). Building microservices: Designing fine-grained systems (2nd ed.). O'Reilly Media.

Shafiei, H., Khonsari, A., & Mousavi, P. (2019). Serverless computing: A survey of opportunities, challenges and applications. arXiv. https://arxiv.org/abs/1911.01296

Shahrad, M., Fonseca, R., Goiri, Í., Chaudhry, G., Batum, P., Cooke, J., Laureano, E., Tresness, C., Russinovich, M., & Bianchini, R. (2020). Serverless in the wild: Characterizing and optimizing the serverless workload at a large cloud provider. arXiv. https://arxiv.org/abs/2003.03423

Wen, J., Chen, Z., Jin, X., & Liu, X. (2022). Rise of the planet of serverless computing: A systematic review. arXiv. https://arxiv.org/abs/2206.12275